

PhD-Level Deep Dive Notes — Backend Foundation Setup to Security

Author: Abdul Mazid

Course: Programming Hero — Level-2, Batch-7, Mission-2 Conceptual Session

Goal: বিশ্বসেরা ইঞ্জিনিয়ার হওয়া — প্রতিটি অক্ষর, প্রতিটি লাইন, কেন, কোথা থেকে, কিভাবে — সব বুঝতে হবে।

Last Updated: 2026-05-22

📁 প্রজেক্ট স্ট্রাকচার (Project Structure) — কেন এভাবে সাজানো?

```
BackendFoundationSetupToSecurity/
├── .env                ← গোপন তথ্য (secrets) রাখার জায়গা
├── package.json        ← প্রজেক্টের "পরিচয়পত্র" + ডিপেন্ডেন্সি তালিকা
├── package-lock.json   ← ডিপেন্ডেন্সির exact version lock
├── tsconfig.json       ← TypeScript কম্পাইলারের নিয়মকানুন
├── node_modules/       ← ডাউনলোড করা সব লাইব্রেরি
├── src/               ← আমাদের লেখা সোর্স কোড
│   ├── app.ts         ← Express অ্যাপ্লিকেশন তৈরি + রাউট ডিফাইন
│   ├── index.ts       ← সার্ভার স্টার্ট (Entry Point)
│   └── config/
│       └── index.ts    ← Environment variable ম্যানেজমেন্ট
└── db/
    └── index.ts        ← ডাটাবেজ কানেকশন
```

🔗 কেন এই স্ট্রাকচার?

এটাকে বলে **Separation of Concerns (SoC)** — সফটওয়্যার ইঞ্জিনিয়ারিংয়ের একটি মূলনীতি। ধরো তুমি একটা হাসপাতাল চালাচ্ছ:

- **app.ts** = হাসপাতালের ডিজাইন (কোন ঘরে কী হবে)
- **index.ts** = হাসপাতাল খোলার সুইচ (দরজা খোলো, বিদ্যুৎ চালু করো)
- **config/** = হাসপাতালের গোপন তথ্য (পাসওয়ার্ড, ঠিকানা)
- **db/** = হাসপাতালের রেকর্ড রুম (ডাটাবেজ)

যদি সব একটা ফাইলে লিখতে — কোড বড় হলে নরক হয়ে যেত। এভাবে আলাদা করলে:

1. **Maintainability** — একটা জিনিস চেঞ্জ করতে পুরো কোড ঘাঁটতে হয় না
2. **Testability** — প্রতিটি অংশ আলাদাভাবে টেস্ট করা যায়
3. **Scalability** — টিমে কাজ করলে একেকজন একেক ফোল্ডারে কাজ করে
4. **Reusability** — **app.ts** কে টেস্টিংয়ে আলাদাভাবে ব্যবহার করা যায়

📄 ফাইল ১: **package.json** — প্রজেক্টের DNA

```
{
  "name": "backendfoundationsetuptosecurity",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "tsx watch ./src/index.ts"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "devDependencies": {
    "@types/express": "^5.0.6",
    "@types/node": "^25.9.1",
    "tsx": "^4.22.3"
  },
  "dependencies": {
    "@neondatabase/serverless": "^1.1.0",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "pg": "^8.21.0"
  }
}
```

লাইন-বাই-লাইন গভীর ব্যাখ্যা:

"name": "backendfoundationsetuptosecurity"

- **কী?** — প্রজেক্টের নাম।
 - **কেন?** — npm (Node Package Manager) প্রতিটি প্যাকেজকে একটি ইউনিক নাম দিয়ে চেনে। যদি তুমি এটা npm-এ পাবলিশ করো, এই নামেই মানুষ `npm install backendfoundationsetuptosecurity` লিখে ইনস্টল করবে।
 - **কোথা থেকে এসেছে?** — যখন তুমি `npm init` কমান্ড চালাও, তখন npm তোমাকে নাম জিজ্ঞেস করে। ডিফল্ট ফোল্ডারের নাম ব্যবহার হয়।
 - **নিয়ম:** lowercase হতে হবে, space থাকা যাবে না, special character সীমিত। কারণ npm registry একটি URL-based system — URL-এ space, uppercase সমস্যা তৈরি করে।
-

"version": "1.0.0"

- **কী?** — **Semantic Versioning (SemVer)** — `MAJOR.MINOR.PATCH` ফরম্যাটে ভার্সন নম্বর।
- **কেন?** — এটা সফটওয়্যার ইন্ডাস্ট্রির স্ট্যান্ডার্ড। Tom Preston-Werner (GitHub-এর সহ-প্রতিষ্ঠাতা) এটি প্রস্তাব করেন।
 - `1` (MAJOR) = বড় পরিবর্তন যা আগের কোড ভেঙে দিতে পারে (breaking change)
 - `0` (MINOR) = নতুন ফিচার যোগ, কিন্তু আগের কোড ভাঙবে না
 - `0` (PATCH) = বাগ ফিক্স

- **বাস্তব উদাহরণ:** React 18.2.0 → 19.0.0 হলো MAJOR change (breaking), 18.2.0 → 18.3.0 হলো MINOR (নতুন ফিচার)।

"description": ""

- **কী?** — প্রজেক্টের সংক্ষিপ্ত বিবরণ।
- **কেন?** — npm search এ মানুষ এই description দেখে বোঝে প্যাকেজ কী করে। খালি রাখলেও চলে, কিন্তু পাবলিশ করলে ভরা উচিত।

"main": "index.js"

- **কী?** — প্রজেক্টের **entry point** — অন্য কেউ যদি তোমার প্যাকেজ `require()` বা `import` করে, কোন ফাইল থেকে শুরু হবে তা বলে।
- **গভীর কথা:** এখানে `index.js` লেখা আছে কিন্তু আমরা TypeScript ব্যবহার করছি। এটা `npm init` এর ডিফল্ট। আমাদের প্রজেক্টে এটা আসলে ব্যবহার হচ্ছে না কারণ আমরা `tsx` দিয়ে সরাসরি `.ts` ফাইল চালাচ্ছি।

"scripts": { "dev": "tsx watch ./src/index.ts" }

- **কী?** — npm scripts — কমান্ড লাইনে `npm run dev` লিখলে এই কমান্ড চলবে।
- **প্রতিটি অংশ ভেঙে দেখা যাক:**

অংশ	ব্যাখ্যা
<code>tsx</code>	TypeScript eXecute — একটি টুল যা TypeScript ফাইল সরাসরি চালাতে পারে কম্পাইল ছাড়াই। ভেতরে এটা <code>esbuild</code> ব্যবহার করে যা Go ভাষায় লেখা, তাই অত্যন্ত দ্রুত।
<code>watch</code>	File watcher মোড — ফাইল সেভ করলেই সার্ভার অটোমেটিক রিস্টার্ট হবে। ভেতরে <code>fs.watch()</code> বা <code>chokidar</code> ব্যবহার করে OS-level file system events শোনে।
<code>./src/index.ts</code>	Entry point ফাইলের path — <code>./</code> মানে current directory থেকে।

- **কেন `tsx`, কেন `ts-node` না?**
 - `ts-node` পুরো TypeScript compiler (`tsc`) ব্যবহার করে — ধীর।
 - `tsx` = `esbuild` ভিত্তিক — ১০০x দ্রুত কারণ `esbuild` Go-তে লেখা, multi-threaded, এবং type-checking skip করে (development-এ speed priority)।

"keywords": []

- **কী?** — npm search এ প্যাকেজ খুঁজে পেতে ব্যবহৃত ট্যাগ/কীওয়ার্ড।
- **কেন খালি?** — আমরা পাবলিশ করছি না, তাই দরকার নেই।

"author": ""

- **কী?** — প্যাকেজ লেখকের নাম।
- **ফরম্যাট:** "Your Name <email@example.com> (https://website.com)" — পূর্ণ ফরম্যাটে লিখলে npm প্রোফাইলে দেখায়।

"license": "ISC"

- **কী?** — **ISC License** — একটি permissive open-source লাইসেন্স।
- **গভীর কথা:** ISC (Internet Systems Consortium) লাইসেন্স MIT লাইসেন্সের মতোই — "যা ইচ্ছা করো, কিন্তু আমাকে দোষ দিও না।" npm ডিফল্টে ISC দেয়। ইন্ডাস্ট্রিতে সবচেয়ে বেশি ব্যবহৃত হয় MIT, Apache-2.0, এবং ISC।

★ "type": "module" — এটা অত্যন্ত গুরুত্বপূর্ণ!

- **কী?** — Node.js-কে বলছে এই প্রজেক্ট **ES Modules (ESM)** ব্যবহার করবে, **CommonJS (CJS)** না।
- **কেন এটা গুরুত্বপূর্ণ?** — **JavaScript মডিউল সিস্টেমের ইতিহাস:**

অধ্যায় ১: সমস্যা (1995-2009)

Brendan Eich 10 দিনে JavaScript বানিয়েছিলেন। সেসময় মডিউল সিস্টেম বলে কিছু ছিল না। সব কোড একটা global scope-এ থাকত। `<script>` ট্যাগ দিয়ে লোড করলে সব variable global হতো — নামের সংঘর্ষ (collision) হতো।

অধ্যায় ২: CommonJS (2009)

Ryan Dahl Node.js বানালেন। সার্ভারে JavaScript চালাতে গেলে মডিউল সিস্টেম লাগবে। তিনি CommonJS স্পেসিফিকেশন ব্যবহার করলেন:

```
// CommonJS - synchronous, runtime-এ কাজ করে
const express = require('express');    // import
module.exports = app;                  // export
```

এটা **synchronous** — মানে একটা ফাইল লোড শেষ না হলে পরের লাইনে যাবে না। সার্ভারে ঠিক আছে (ফাইল disk থেকে পড়ে), কিন্তু ব্রাউজারে?

অধ্যায় ৩: ES Modules (2015 — ECMAScript 2015 / ES6)

TC39 কমিটি (JavaScript-এর ভাষা কমিটি) একটি **অফিসিয়াল মডিউল সিস্টেম** তৈরি করল:

```
// ESM - static, compile-time-এ analyze করা যায়
import express from 'express';    // import
export default app;                // export
```

ESM কেন ভালো?

- **Static analysis** — import/export কম্পাইল টাইমে বোঝা যায়, তাই **tree-shaking** (অব্যবহৃত কোড সরানো) সম্ভব।
- **Asynchronous** — ব্রাউজারে network থেকে ফাইল আনতে হয়, async না হলে পেজ hang করত।
- **Standardized** — সব জায়গায় একই syntax (ব্রাউজার + সার্ভার)।

"type": "module" লিখলে Node.js বোঝে: .js ফাইলগুলো ESM হিসেবে treat করো (import/export ব্যবহার করো, require() না)।

"devDependencies" — শুধু Development-এ লাগে

```
"devDependencies": {
  "@types/express": "^5.0.6",
  "@types/node": "^25.9.1",
  "tsx": "^4.22.3"
}
```

devDependencies vs dependencies — পার্থক্য কী?

	dependencies	devDependencies
কখন লাগে?	Production-এ (সার্ভারে চলার সময়)	শুধু development-এ (কোড লেখার সময়)
Install কমান্ড	<code>npm install express</code>	<code>npm install -D @types/express</code>
Production-এ যায়?	হ্যাঁ	না (<code>npm install --production</code> করলে skip হয়)

"@types/express": "^5.0.6"

- **কী?** — Express-এর TypeScript type definitions।
- **কেন?** — Express মূলত JavaScript-এ লেখা। TypeScript বুঝবে না `req`, `res`, `app` এর কী কী property/method আছে। `@types/express` ফাইলে `.d.ts` (declaration files) থাকে যা TypeScript-কে বলে:

```
// এরকম definition থাকে .d.ts ফাইলে:
interface Request {
  body: any;
  params: Record<string, string>;
  query: Record<string, string>;
  // ... আরও অনেক
}
```

- **@types কোথা থেকে এসেছে?** — **DefinitelyTyped** — পৃথিবীর সবচেয়ে বড় TypeScript type repository। হাজার হাজার contributor JS লাইব্রেরির জন্য type লেখেন।
- **@ মানে কী?** — npm **scoped package**। `@types` হলো scope name, `express` হলো package name। Scope দিয়ে related packages গ্রুপ করা যায়।

```
"@types/node": "^25.9.1"
```

- **কী?** — Node.js built-in modules-এর (fs, path, http, process, etc.) TypeScript type definitions।
 - **কেন?** — ছাড়া TypeScript `process.env`, `console.log`, `Buffer` ইত্যাদি চিনবে না।
-

```
"tsx": "^4.22.3"
```

- **কী?** — TypeScript eXecute — `.ts` ফাইল সরাসরি চালায় কম্পাইল ছাড়া।
 - **কিভাবে কাজ করে?** — ভেতরে `esbuild` ব্যবহার করে TypeScript → JavaScript রূপান্তর করে, তারপর Node.js-এ চালায়। `esbuild` Go ভাষায় লেখা বলে C/C++ এর কাছাকাছি speed পায়।
 - **কেন devDependency?** — Production-এ আমরা আগে compile করে `.js` বানিয়ে deploy করব, তাই `tsx` production-এ লাগবে না।
-

```
^ (caret) চিহ্নের অর্থ কী? — "^5.0.6"
```

- **SemVer Range** — `^` মানে MAJOR version ঠিক থাকবে, MINOR ও PATCH up হতে পারবে।
 - `^5.0.6` → `>=5.0.6` এবং `<6.0.0` — যেকোনো `5.x.x` ভার্সন চলবে।
 - **কেন?** — MINOR/PATCH আপডেটে breaking change হওয়ার কথা না (SemVer-এর নিয়ম অনুযায়ী), তাই নিরাপদে আপডেট নেওয়া যায়।
 - **অন্যান্য চিহ্ন:**
 - `~5.0.6` = PATCH-ই শুধু up হবে (`>=5.0.6`, `<5.1.0`)
 - `5.0.6` = exact version, কোনো আপডেট না
 - `*` = যেকোনো version (বিপজ্জনক!)
-

```
"dependencies" — Production-এ লাগে
```

```
"dependencies": {
  "@neondatabase/serverless": "^1.1.0",
  "dotenv": "^17.4.2",
  "express": "^5.2.1",
  "pg": "^8.21.0"
}
```

```
"@neondatabase/serverless": "^1.1.0"
```

- **কী?** — **Neon** — একটি serverless PostgreSQL ডাটাবেজ সার্ভিস। এই প্যাকেজ HTTP/WebSocket দিয়ে Neon-এর ডাটাবেজে কানেক্ট করে।
- **কেন serverless?** — Traditional PostgreSQL connection TCP socket দিয়ে কাজ করে, যা serverless environment-এ (Vercel, Cloudflare Workers) কাজ করে না কারণ সেখানে persistent TCP connection রাখা যায় না। Neon HTTP-based protocol ব্যবহার করে এই সমস্যা সমাধান করে।
- **neon()** ফাংশন কী করে? — একটি SQL template tag function তৈরি করে। তুমি এভাবে query লিখতে পারো: `sql`SELECT * FROM users WHERE id = ${userId}`` — এটা **SQL injection** থেকে স্বয়ংক্রিয়ভাবে protect করে (parameterized query)।

"dotenv": "^17.4.2"

- **কী?** — `.env` ফাইল থেকে environment variables পড়ে `process.env`-এ load করে।
 - **কেন দরকার?** — **Twelve-Factor App methodology** (Heroku-এর co-founder Adam Wiggins তৈরি করেছেন) বলে: "Config-কে environment-এ store করো।" মানে পাসওয়ার্ড, API key, ডাটাবেজ URL কখনো কোডে হার্ডকোড করবে না — `.env` ফাইলে রাখো এবং `.gitignore`-এ যোগ করো যেন Git-এ push না হয়।
 - **কিভাবে কাজ করে?**
 1. `.env` ফাইল পড়ে (Node.js `fs.readFileSync`)
 2. প্রতিটি লাইন parse করে `KEY=VALUE` ফরম্যাটে
 3. `process.env.KEY = VALUE` সেট করে
 4. এরপর তোমার কোডে `process.env.PORT` লিখলে value পাবে
-

"express": "^5.2.1"

- **কী?** — **Express.js** — Node.js-এর সবচেয়ে জনপ্রিয় **web framework**।
- **ইতিহাস:** TJ Holowaychuk ২০১০ সালে তৈরি করেন। Sinatra (Ruby framework) দ্বারা অনুপ্রাণিত। "Minimalist, unopinionated" — মানে তোমাকে বেশি কিছু জোর করে না, তুমি যেভাবে চাও সেভাবে সাজাতে পারো।
- **Express কী করে?**
 1. HTTP server তৈরি করে (Node.js-এর built-in `http` module-এর উপর)
 2. **Routing** — URL pattern অনুযায়ী সঠিক function-এ request পাঠায়
 3. **Middleware** — request ও response-এর মাঝে processing chain তৈরি করে
 4. Response পাঠানোর সুবিধাজনক methods দেয় (`res.json()`, `res.send()`, etc.)
- **Express ছাড়া যদি করতে:**

```
// Pure Node.js – Express ছাড়া
import http from 'http';
const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Hello World');
  }
});
server.listen(5000);
```

Express এই boilerplate কোডকে সুন্দর, readable, scalable করে।

"pg": "^8.21.0"

- **কী?** — **node-postgres** — Node.js থেকে PostgreSQL ডাটাবেজে কানেক্ট করার জন্য native driver।
- **কেন?** — Neon serverless driver HTTP দিয়ে কাজ করে (lightweight), কিন্তু `pg` TCP socket দিয়ে কাজ করে (traditional, full-featured)। দুটো রাখার কারণ হলো — local development-এ `pg` ভালো, production/serverless-এ `@neondatabase/serverless` ভালো।

- **কিভাবে কাজ করে?** — TCP connection pool তৈরি করে, SQL query পাঠায়, result parse করে JavaScript object হিসেবে return করে। libpq (PostgreSQL-এর C client library) এর JavaScript implementation।

ফাইল ২: `tsconfig.json` — TypeScript কম্পাইলারের সংবিধান

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "esnext",
    "moduleResolution": "bundler",
    "target": "esnext",
    "types": [],
    "sourceMap": true,
    "declaration": true,
    "declarationMap": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "strict": true,
    "jsx": "react-jsx",
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "noUncheckedSideEffectImports": true,
    "moduleDetection": "force",
    "skipLibCheck": true
  }
}
```

প্রতিটি option গভীরভাবে:

"rootDir": "./src"

- **কী?** — TypeScript-কে বলছে: "আমার সোর্স কোড `./src` ফোল্ডারে আছে।"
- **কেন?** — কম্পাইল করলে `src/` ফোল্ডারের structure `dist/` ফোল্ডারে mirror হবে। `rootDir` না দিলে TypeScript নিজে সবচেয়ে উপরের `.ts` ফাইল খুঁজে root ঠিক করে — এতে unexpected ফোল্ডার structure তৈরি হতে পারে।

"outDir": "./dist"

- **কী?** — কম্পাইল করা JavaScript ফাইল `./dist` ফোল্ডারে যাবে।
- **কেন `dist`?** — Convention: `dist` = "distribution" — production-ready কোড। অন্যান্য নাম: `build`, `out`, `lib`।

★ **"module": "esnext"**

- **কী?** — কম্পাইল করা JavaScript-এ কোন module system ব্যবহার হবে তা বলে।
- **esnext** **মানে কী?** — সর্বশেষ ECMAScript module standard ব্যবহার করো। মানে `import/export` যেমন আছে তেমনই রাখো, CommonJS-এ (`require/module.exports`) রূপান্তর করো না।
- **অন্যান্য option:**
 - **commonjs** — `require()/module.exports` এ রূপান্তর করবে
 - **es2020** — ES2020 standard-এর module features
 - **nodenext** — Node.js-specific ESM behavior

★ "moduleResolution": "bundler"

- **কী?** — TypeScript কিভাবে `import` statement-এর path resolve করবে (ফাইল খুঁজে পাবে) তা বলে।
- **bundler** **মানে কী?** — Modern bundler (Vite, esbuild, webpack) যেভাবে path resolve করে সেভাবে করো:
 - `import config from './config' → ./config/index.ts` খুঁজবে
 - Extension ছাড়াই import করা যাবে
 - `package.json`-এর `exports` field সম্মান করবে
- **কেন bundler?** — আমরা `tsx` (esbuild-based) ব্যবহার করছি, তাই bundler-style resolution দরকার।
- **অন্যান্য option:**
 - **node** — Node.js CommonJS-style resolution
 - **nodenext** — Node.js ESM-style (extension দিতে হয়: `./config/index.js`)
 - **classic** — পুরনো TypeScript resolution (আর ব্যবহার হয় না)

"target": "esnext"

- **কী?** — কম্পাইল করা JavaScript কোন ECMAScript version-এ হবে।
- **esnext** **মানে কী?** — সর্বশেষ JavaScript features ব্যবহার করো, পুরনো syntax-এ রূপান্তর করো না।
- **উদাহরণ:** `target: "es5"` দিলে `const → var`, arrow function `→ function` হয়ে যেত। **esnext** দিলে যেমন আছে তেমনই থাকে, কারণ আমরা latest Node.js চালাচ্ছি যা সব নতুন feature support করে।

"types": []

- **কী?** — কোন global type packages auto-include হবে তা বলে।
- **[] (খালি array)** **মানে কী?** — কোনো global type auto-include করো না। এটা intentional — তুমি explicitly যেখানে দরকার সেখানে import করবে।
- **কেন?** — Precision। `["node"]` দিলে সব Node.js types globally available হতো। খালি রেখে তুমি নিশ্চিত করছো কোন type কোথা থেকে আসছে তা তুমি জানো।

"sourceMap": true

- **কী?** — `.js.map` ফাইল তৈরি করে যা compiled JavaScript-কে original TypeScript-এ map করে।
- **কেন?** — Debugging-এর জন্য। ব্রাউজার বা Node.js-এ error হলে stack trace দেখাবে TypeScript ফাইলের লাইন নম্বর, JavaScript-এর না। Chrome DevTools এই map ফাইল পড়ে TypeScript কোড দেখায়।
- **কিভাবে কাজ করে?** — Source map একটি JSON ফাইল যাতে `mappings` field থাকে — Base64 VLQ (Variable-Length Quantity) encoded string যা প্রতিটি generated line/column কে original line/column-এ map করে।

"declaration": true

- **কী?** — `.d.ts` (declaration) ফাইল তৈরি করে।
- **কেন?** — তোমার কোড যদি অন্যরা TypeScript-এ ব্যবহার করে, তাদের type information লাগবে। `.d.ts` ফাইলে শুধু type থাকে, implementation থাকে না:

```
// app.d.ts (generated)
declare const app: Application;
export default app;
```

"declarationMap": true

- **কী?** — `.d.ts.map` ফাইল তৈরি করে যা declaration file-কে original source-এ map করে।
- **কেন?** — IDE-তে "Go to Definition" করলে `.d.ts` ফাইলে না গিয়ে সরাসরি original `.ts` source-এ যাবে।

★ "noUncheckedIndexedAccess": true

- **কী?** — Array বা object index access-এ `undefined` possibility যোগ করে।
- **উদাহরণ:**

```
const arr = [1, 2, 3];
const val = arr[5]; // false ছাড়া: number, true দিলে: number | undefined
```

- **কেন?** — JavaScript-এ array out-of-bounds access `undefined` return করে, error দেয় না। এই option TypeScript-কে বাধ্য করে সেই `undefined` possibility handle করতে — runtime error আগেই catch হয়।

"exactOptionalPropertyTypes": true

- **কী?** — Optional property (?) -তে explicitly `undefined` assign করা আলাদা জিনিস হিসেবে ধরে।
- **উদাহরণ:**

```
interface User {
  name: string;
  age?: number; // property থাকতেও পারে, না-ও থাকতে পারে
}
// false: { age: undefined } ✅ allowed
// true: { age: undefined } ❌ error! "না থাকা" আর "undefined থাকা" আলাদা
```

- **কেন?** — Database-এ `NULL` value insert করা আর column না থাকা সম্পূর্ণ আলাদা concept। এই strictness সেই পার্থক্য enforce করে।

★ "strict": true

- **কী?** — একসাথে অনেকগুলো strict checking option চালু করে:
 - `strictNullChecks` — `null/undefined` আলাদাভাবে handle করতে বাধ্য
 - `strictFunctionTypes` — function parameter types strictly check
 - `strictBindCallApply` — `bind`, `call`, `apply` এর argument type check
 - `strictPropertyInitialization` — class property constructor-এ initialize করতে বাধ্য
 - `noImplicitAny` — type না দিলে `any` ধরে না, error দেয়
 - `noImplicitThis` — `this`-এর type বুঝতে না পারলে error
 - `alwaysStrict` — প্রতিটি ফাইলে `"use strict"` যোগ করে
 - `useUnknownInCatchVariables` — catch block-এ error `unknown` type হবে, `any` না
- **কেন?** — "বিশ্বসেরা ইঞ্জিনিয়ার" হতে চাইলে strictest possible settings ব্যবহার করো। Loose settings দিলে bugs production-এ ধরা পড়ে, strict দিলে development-এই catch হয়।

"jsx": "react-jsx"

- **কী?** — JSX syntax কিভাবে handle করবে তা বলে।
- **কেন backend-এ JSX?** — এই প্রজেক্টে দরকার নেই, কিন্তু `tsconfig` template-এ ছিল। যদি ভবিষ্যতে SSR (Server-Side Rendering) বা React Email template ব্যবহার করো তাহলে কাজে লাগবে।
- **react-jsx মানে কী?** — React 17+ এর new JSX transform ব্যবহার করবে (`import React` দরকার নেই)।

★ "verbatimModuleSyntax": true

- **কী?** — `import type` এবং `import` কে strictly আলাদা করে।
- **উদাহরণ:**

```
import type { Request } from 'express'; // ✅ শুধু type – compiled JS-এ এটা থাকবে না
import express from 'express';          // ✅ value – compiled JS-এ থাকবে
```

- **কেন?** — Type-only imports compiled JavaScript-এ যাওয়ার দরকার নেই। এই option enforce করে যে তুমি clearly বলবে কোনটা type আর কোনটা value — bundler-এর জন্য tree-shaking সহজ হয়।

"isolatedModules": true

- **কী?** — প্রতিটি ফাইল independently transpile করা যাবে কিনা তা check করে।
- **কেন?** — `tsx/esbuild/Babel` প্রতিটি ফাইল আলাদাভাবে process করে (full program analysis করে না)। কিছু TypeScript feature (যেমন `const enum`, `namespace` merging) single-file transpilation-এ কাজ করে না। এই option সেগুলো ব্যবহার করলে error দেয়।

"noUncheckedSideEffectImports": true

- **কী?** — Side-effect-only imports (যেমন `import './polyfill'`) এ ফাইল exist করে কিনা check করে।

- **কেন?** — `import './missing-file'` লিখলে normally কোনো error হয় না (TypeScript ধরে নেয় side-effect import)। এই option চালু করলে ফাইল না থাকলে error দেবে।

"moduleDetection": "force"

- **কী?** — সব ফাইলকে module হিসেবে ধরে (script না)।
- **কেন?** — JavaScript-এ `import/export` না থাকলে ফাইল "script" হিসেবে ধরা হয় (global scope)। "force" দিলে সব ফাইল module হবে, এমনকি `import/export` না থাকলেও। এটা accidental global scope pollution রোধ করে।

"skipLibCheck": true

- **কী?** — `.d.ts` (declaration) ফাইলের type checking skip করে।
- **কেন?** — `node_modules`-এর হাজার হাজার `.d.ts` ফাইল check করলে compilation অনেক ধীর হয়ে যায়। তোমার কোডের type checking হবে, কিন্তু third-party library-র declaration file-এর error ignore হবে। Production-grade প্রজেক্টে প্রায় সবাই এটা `true` রাখে।

ফাইল ৩: `.env` — গোপন তথ্যের ভল্ট

```
PORT = 5000
DATABASE_URL = postgresql://neondb_owner:npg_fVX8N2ivPrmj@ep-dawn-grass-aoh6l44w-
pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?
sslmode=require&channel_binding=require
```

লাইন-বাই-লাইন:

PORT = 5000

- **কী?** — সার্ভার কোন port-এ চলবে তা বলে।
- **Port কী?** — কম্পিউটারের networking-এ port হলো একটি **logical address** (0-65535)। IP address দিয়ে কম্পিউটার চেনা যায়, port দিয়ে কম্পিউটারের কোন application/service চেনা যায়।
 - Port 80 = HTTP
 - Port 443 = HTTPS
 - Port 5432 = PostgreSQL
 - Port 5000 = আমাদের Express সার্ভার (custom)
- **কেন 5000?** — Convention। Well-known ports (0-1023) OS-reserved, registered ports (1024-49151) software-specific। 3000, 5000, 8080 development-এ বেশি ব্যবহৃত হয়।

DATABASE_URL = postgresql://... — PostgreSQL Connection String ভেঙে ভেঙে বোঝা

```
postgresql://neondb_owner:npg_fVX8N2ivPrmj@ep-dawn-grass-aoh6l44w-pooler.c-2.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

এটা একটি **URI (Uniform Resource Identifier)** — RFC 3986 standard অনুযায়ী:

```
scheme://username:password@host/database?parameters
```

অংশ	মান	ব্যাখ্যা
Scheme	postgresql://	Protocol বলছে — PostgreSQL ডাটাবেজে কানেক্ট হবে
Username	neondb_owner	ডাটাবেজ ইউজারনেম — কার পক্ষ থেকে query চলবে
Password	npg_fVX8N2ivPrmj	⚠ গোপন! এটা GitHub-এ push করলে হ্যাকাররা তোমার DB access পেয়ে যাবে
Host	ep-dawn-grass-aoh6l44w-pooler.c-2.ap-southeast-1.aws.neon.tech	Neon-এর সার্ভারের ঠিকানা। ap-southeast-1 = Singapore AWS region
Database	neondb	কোন database-এ কানেক্ট হবে (একটি PostgreSQL server-এ অনেক database থাকতে পারে)
sslmode=require	SSL/TLS encryption বাধ্যতামূলক — data encrypt হয়ে যাবে transit-এ	
channel_binding=require	SCRAM-SHA-256 channel binding — man-in-the-middle attack প্রতিরোধ	

⚠ নিরাপত্তা সতর্কতা: .env ফাইল .gitignore-এ যোগ করো! Git-এ push করলে credential leak হবে। তোমার .env ফাইলে real password আছে — এটা expose হলে Neon ডাটাবেজের পুরো access চলে যাবে।

📄 ফাইল 8: src/config/index.ts — Configuration Management

```
import dotenv from 'dotenv';
import { env } from 'process';

dotenv.config({ quiet: true });

const config = {
  port: env.PORT as string,
  database_url: env.DATABASE_URL as string,
```

```
};

export default config;
```

লাইন ১: `import dotenv from 'dotenv';`

- **কী হচ্ছে?** — `dotenv` প্যাকেজ থেকে **default export** import করছি।
- **import ... from '...' কিভাবে কাজ করে?**
 1. Nodejs 'dotenv' string পায়
 2. `node_modules/dotenv/` ফোল্ডারে যায়
 3. `package.json`-এর "main" বা "exports" field দেখে কোন ফাইল entry point
 4. সেই ফাইলের `export default` বা `module.exports` কে `dotenv` variable-এ রাখে
- **default import কী?** — একটি module-এর primary/main export। প্রতিটি module-এ শুধু একটি default export থাকতে পারে। Named export অনেকগুলো থাকতে পারে:

```
// default export
import dotenv from 'dotenv';      // নাম যেকোনো কিছু দেওয়া যায়
// named export
import { config } from 'dotenv';  // নাম exact match হতে হবে
```

লাইন ২: `import { env } from 'process';`

- **কী হচ্ছে?** — Nodejs-এর built-in `process` module থেকে `env` object destructure করে import করছি।
- **process কী?** — Nodejs-এর **global object** যা running process সম্পর্কে তথ্য ও নিয়ন্ত্রণ দেয়:
 - `process.env` — Environment variables (key-value pairs)
 - `process.argv` — Command line arguments
 - `process.pid` — Process ID
 - `process.exit()` — Process বন্ধ করে
 - `process.cwd()` — Current working directory
- **{ env } কী?** — **Destructuring import** — `process` object-এর `env` property-কে সরাসরি নিচ্ছি। `process.env` এর বদলে শুধু `env` লিখলেই চলবে।
- **process.env কী?** — Operating System-এর environment variables-এর একটি object। OS-level-এ `SET PORT=5000` (Windows) বা `export PORT=5000` (Linux) করলে `process.env.PORT` এ পাওয়া যায়। `dotenv` এই mechanism ব্যবহার করে `.env` ফাইলের variables inject করে।

লাইন ৪: `dotenv.config({ quiet: true });`

- **কী হচ্ছে?** — `.env` ফাইল পড়ে সেখানকার variables `process.env`-এ load করছে।
- **config() কী করে ভেতরে?**
 1. Current directory-তে `.env` ফাইল খোঁজে
 2. `fs.readFileSync('.env', 'utf-8')` দিয়ে পড়ে
 3. প্রতিটি লাইন `KEY=VALUE` parse করে
 4. `process.env[KEY] = VALUE` সেট করে

- **{ quiet: true } কী?** — .env ফাইল না পাওয়া গেলে console-এ warning দেখাবে না। Production-এ .env ফাইল নাও থাকতে পারে (environment variables সরাসরি OS-level-এ set করা হয়), তাই warning অপ্রয়োজনীয়।

লাইন ৬-১০: Config Object

```
const config = {
  port: env.PORT as string,
  database_url: env.DATABASE_URL as string,
};
```

- **কী হচ্ছে?** — একটি config object তৈরি করছি যেখানে environment variables centralized।
- **const কেন, let না?** — const মানে variable re-assign করা যাবে না। config একবার তৈরি হলে বদলানো উচিত না — **immutability principle**।
- **as string কী?** — **Type Assertion** — TypeScript-কে বলছি: "আমি নিশ্চিত এটা string, তুমি trust করো।"
 - process.env.PORT এর type হলো **string | undefined** — কারণ environment variable set না থাকতেও পারে
 - as string দিয়ে undefined possibility remove করছি
 - ⚠ **সতর্কতা:** এটা runtime safety দেয় না! যদি .env ফাইলে PORT না থাকে, config.port হবে undefined — কিন্তু TypeScript error দেবে না কারণ তুমি as string দিয়ে বলেছো "এটা string!" Production-grade code-এ validation করা উচিত:

```
const port = env.PORT;
if (!port) throw new Error('PORT environment variable is required');
```

লাইন ১২: export default config;

- **কী হচ্ছে?** — config object-কে এই module-এর **default export** হিসেবে export করছি।
- **কেন default?** — এই ফাইল থেকে শুধু একটাই জিনিস export হচ্ছে, তাই default export যুক্তিসঙ্গত। অন্য ফাইলে import config from './config' লিখলেই পাবে।
- **কেন ./config এ /index.ts লিখতে হয় না?** — Node.js (এবং bundler) convention: ফোল্ডার import করলে সেই ফোল্ডারের index.ts/index.js automatically resolve হয়। এটাকে বলে **barrel file pattern**।

ফাইল ৫: src/app.ts — Express Application

```
import express, { type Application, type Request, type Response } from 'express';

const app: Application = express();

app.get('/', (req: Request, res: Response) => {
  res.send("Hello world , I'm Abdul Mazid From express");
});
```

```
export default app;
```

লাইন ১: `import express, { type Application, type Request, type Response } from 'express';`

এটা একটি **mixed import** — একসাথে default এবং named imports:

Part	Type	ব্যাখ্যা
<code>express</code>	Default import (value)	Express-এর main function — call করলে app তৈরি হয়
<code>type Application</code>	Named import (type-only)	Express Application-এর TypeScript type/interface
<code>type Request</code>	Named import (type-only)	HTTP Request object-এর type
<code>type Response</code>	Named import (type-only)	HTTP Response object-এর type

- **type keyword কেন import-এ?** — `verbatimModuleSyntax: true` চালু আছে (tsconfig-এ), তাই explicitly বলতে হবে কোনটা type আর কোনটা value। `type` দিয়ে import করলে compiled JavaScript-এ এই imports থাকবে না (কারণ JavaScript-এ type বলে কিছু নেই)।
- **express function কী?** — এটা একটি **factory function**। Call করলে একটি `Application` object return করে। ভেতরে এটা Node.js-এর `http.createServer()` এর wrapper। Express app আসলে একটি request handler function: `(req, res) => void`।

লাইন ৩: `const app: Application = express();`

- **Application type কী?** — Express application-এর interface। এতে যা আছে:
 - `.get(), .post(), .put(), .delete()` — HTTP method handlers
 - `.use()` — middleware যোগ করা
 - `.listen()` — সার্ভার চালু করা
 - `.set(), .get()` — app settings
 - ...এবং আরো অনেক method
- **: Application লেখা কি বাধ্যতামূলক?** — না! TypeScript **type inference** করতে পারে — `express()` return type দেখে বুঝে নেয়। কিন্তু explicitly লেখা **self-documenting code** — পড়ে বোঝা সহজ, IDE support ভালো পাওয়া যায়।
- **express() call করলে ভেতরে কী হয়?**
 1. একটি app function তৈরি হয় (`function(req, res, next) { ... }`)
 2. এই function-এ Express-এর সব method (`.get, .post, .use`, etc.) attach করে
 3. Internal request pipeline/middleware stack initialize করে
 4. Router তৈরি করে

লাইন ৫-৭: Route Handler

```
app.get('/', (req: Request, res: Response) => {
  res.send("Hello world , I'm Abdul Mazid From express");
});
```



```
});
```

`app.get('/', ...)`

- **app.get()** কী করে? — HTTP GET method-এর জন্য **route handler** register করে।
- **HTTP Methods** কী?

Method	কাজ	উদাহরণ
GET	Data চাওয়া (Read)	ওয়েবসাইট দেখা, API থেকে data fetch
POST	নতুন data তৈরি (Create)	Form submit, নতুন user তৈরি
PUT	পুরো data আপডেট (Replace)	User profile সম্পূর্ণ update
PATCH	আংশিক আপডেট	শুধু email change
DELETE	Data মোছা	Account delete

এগুলো **HTTP/1.1 specification (RFC 7231)** এ সংজ্ঞায়িত।

- **'/'** কী? — **Route path/pattern**। URL-এর path অংশ। `http://localhost:5000/` এ `/` হলো root path। ব্রাউজারে `http://localhost:5000` লিখলে যে request যায় তার path হয় `/`।

`(req: Request, res: Response) => { ... }`

- **কী?** — **Arrow function** — route handler/callback।
- **=> (arrow function) কোথা থেকে এসেছে?**
 - ES6 (2015) এ introduce হয়েছে
 - C#-এর lambda expressions এবং CoffeeScript থেকে অনুপ্রাণিত
 - Regular function-এর চেয়ে পার্থক্য:
 1. **Lexical this binding** — নিজের **this** নেই, parent-এর **this** ব্যবহার করে
 2. **Shorter syntax** — **function** keyword লাগে না
 3. **Cannot be used as constructor** — **new** দিয়ে call করা যায় না
- **req: Request** কী? — HTTP Request object:
 - `req.body` — POST request-এর data
 - `req.params` — URL parameters (`/users/:id` → `req.params.id`)
 - `req.query` — Query string (`?page=2` → `req.query.page`)
 - `req.headers` — HTTP headers (Content-Type, Authorization, etc.)
 - `req.method` — 'GET', 'POST', etc.
 - `req.url` — Request URL
- **res: Response** কী? — HTTP Response object:
 - `res.send()` — Response পাঠায় (string/Buffer/object)
 - `res.json()` — JSON response পাঠায়

- `res.status()` — HTTP status code set করে
- `res.redirect()` — অন্য URL-এ redirect করে
- `res.cookie()` — Cookie set করে

```
res.send("Hello world , I'm Abdul Mazid From express");
```

- **`res.send()` কী করে ভেতরে?**

1. **Content-Type header set করে** — string দিলে `text/html`, Buffer দিলে `application/octet-stream`, object দিলে `application/json`
2. **Content-Length header set করে** — response body-র byte size
3. **ETag header generate করে** — caching-এর জন্য (response content-এর hash)
4. **Response body write করে** — Node.js-এর `res.write()` + `res.end()` call করে
5. HEAD request হলে শুধু headers পাঠায়, body না

লাইন ৯: `export default app;`

- **কী হচ্ছে?** — `app` object-কে export করছি যেন `index.ts` (server starter) এটা import করে ব্যবহার করতে পারে।
- **কেন `app` আলাদা ফাইলে?** — **Testability!** Unit test-এ `app` import করে `supertest` দিয়ে test করা যায়, সার্ভার start না করেই:

```
import request from 'supertest';
import app from './app';
test('GET /', async () => {
  const res = await request(app).get('/');
  expect(res.status).toBe(200);
});
```

ফাইল ৬: `src/index.ts` — Server Entry Point

```
import app from './app';
import config from './config';

const main = async () => {
  console.log(config.database_url);
  app.listen(config.port, () => {
    console.log(`Server is running on port ${config.port}`);
  });
};

main();
```

লাইন ১-২: Imports

```
import app from './app';
import config from './config';
```

- **'./app'** — Relative path import. `./` মানে current directory (`src/`)। TypeScript + bundler resolution `./app` → `./app.ts` resolve করে।
- **'./config'** — `./config` ফোল্ডারের `index.ts` resolve হয় (barrel file pattern)।

লাইন ৪: `const main = async () => { ... }`

- **async keyword কী?** — এই function **asynchronous** — মানে এর ভেতরে `await` keyword ব্যবহার করা যায়।
- **কেন async?** — ভবিষ্যতে database connection (`await`) বা অন্য asynchronous কাজ এখানে হবে। এখনো `await` ব্যবহার হচ্ছে না, কিন্তু structure ready রাখা হয়েছে।
- **async function কী return করে?** — সবসময় একটি **Promise** return করে। `async () => 5` আসলে return করে `Promise.resolve(5)`।
- **Promise কী?** — JavaScript-এ asynchronous operation-এর ফলাফল represent করে। তিনটি অবস্থা:
 1. **Pending** — এখনো শেষ হয়নি
 2. **Fulfilled** — সফল হয়েছে, result আছে
 3. **Rejected** — ব্যর্থ হয়েছে, error আছে
- **কেন main function-এ wrap করলাম? সরাসরি code লিখলে হতো না?**
 - **Top-level await** ESM-এ support করে, কিন্তু সব environment-এ নয়।
 - **main function pattern** দিলে:
 1. Error handling সহজ: `main().catch(console.error)` দিয়ে unhandled errors ধরা যায়
 2. Graceful shutdown implement করা যায়
 3. Code organization ভালো হয়

লাইন ৫: `console.log(config.database_url);`

- **কী?** — Database URL console-এ print করছে — verify করতে যে `.env` ঠিকভাবে load হয়েছে।
- **⚠ Production-এ এটা রাখা বিপজ্জনক!** — Database password log-এ দেখা যাবে। Development-এ debugging-এর জন্য ঠিক আছে, কিন্তু deploy-এর আগে সরিয়ে ফেলো বা sensitive part mask করো।
- **console.log() কিভাবে কাজ করে?** — Node.js-এ `console.log` আসলে `process.stdout.write()` call করে — অর্থাৎ standard output stream-এ লেখে। Terminal সেই stream পড়ে screen-এ দেখায়।

লাইন ৬-৮: Server Start

```
app.listen(config.port, () => {
  console.log(`Server is running on port ${config.port}`);
});
```

app.listen(port, callback)

- **কী করে ভেতরে?**
 1. Node.js-এর `http.createServer(app)` call করে — একটি HTTP server তৈরি হয়
 2. `server.listen(port)` call করে — নির্দিষ্ট port-এ listening শুরু করে
 3. OS-এর **TCP/IP stack**-এ একটি **socket** তৈরি হয় — এই socket incoming connections-এর জন্য অপেক্ষা করে
 4. Port bind সফল হলে callback function call হয়
- **Callback () => { ... } কী?** — Server successfully start হলে এই function execute হবে। এটা **event-driven programming** — "এটা হলে এটা করো" pattern।
- **Template Literal `Server is running on port \${config.port}`**
 - **কী?** — ES6 template string — backtick (` ) দিয়ে string তৈরি, `${}` দিয়ে expression embed করা যায়।
 - **কেন?** — String concatenation-এর (`'text' + variable + 'text'`) চেয়ে পড়তে সহজ, multi-line string-ও সম্ভব।

লাইন ১১: `main();`

- **কী?** — `main` function-কে call (invoke) করছি — সার্ভার শুরু হচ্ছে!
- **কী হয় এই call-এ?**
 1. `main()` call হয়
 2. `config.database_url` print হয়
 3. `app.listen()` call হয় — সার্ভার port-এ bind হয়
 4. Bind সফল হলে callback চলে — "Server is running..." print হয়
 5. Node.js **event loop**-এ ঢুকে যায় — incoming HTTP requests-এর জন্য অপেক্ষা করতে থাকে

ফাইল ৭: `src/db/index.ts` — Database Connection

```
import { neon } from '@neondatabase/serverless';
import config from '../config';

export const sql = neon(config.database_url);
```

লাইন ১: `import { neon } from '@neondatabase/serverless';`

- `{ neon }` — Named import। `@neondatabase/serverless` package থেকে `neon` নামের function import করছি।
- **neon function কী?** — একটি **factory function** যা database connection string নিয়ে একটি **SQL tagged template function** return করে।

লাইন ২: `import config from '../config';`

- `'../config'` — `..` মানে parent directory-তে যাও। `db/` থেকে `../` → `src/`, তারপর `config/` → `config/index.ts`।
- **Path resolution chain:** `src/db/index.ts` → `../config` → `src/config` → `src/config/index.ts`

লাইন 8: `export const sql = neon(config.database_url);`

- **`neon(config.database_url)` কী করে?**

1. Connection string parse করে (username, password, host, database বের করে)
2. একটি function return করে যেটা **tagged template literal** হিসেবে কাজ করে
3. এই function SQL query execute করতে পারে:

```
// ব্যবহারের উদাহরণ:
const users = await sql`SELECT * FROM users WHERE age > ${18}`;
```

4. **Important:** `${18}` directly SQL-এ বসে না — এটা **parameterized query** হিসেবে পাঠানো হয়। এটা **SQL Injection** attack থেকে protect করে।

- **Tagged Template Literal কী?**

```
// normal template literal:
const greeting = `Hello ${name}`;

// tagged template literal:
const result = sql`SELECT * FROM users WHERE id = ${userId}`;
// sql() function-কে দুটো argument যায়:
//   strings: ['SELECT * FROM users WHERE id = ', '']
//   values: [userId]
// function নিজে decide করে কিভাবে combine করবে
```

এটা ES6-এর একটি powerful feature — custom string processing-এর জন্য।

- **`export const` কেন `export default` না?**
 - **Named export** (`export const sql`) — import-এ exact নাম ব্যবহার করতে হবে: `import { sql } from '../db'`
 - **কেন?** — এই ফাইল থেকে ভবিষ্যতে আরো exports হতে পারে (যেমন `pool`, `transaction` function)। Named export scalable।

✖ সব মিলিয়ে Application Flow — কোডটি কিভাবে চলে?

```
[npm run dev]
↓
[tsx watch ./src/index.ts] ← tsx, TypeScript ফাইল সরাসরি চালায়
↓
```

```

[src/index.ts] ← Entry point
↓
├─ import app from './app'
│   ↓
│   [src/app.ts]
│   │   └─ import express → Express app তৈরি
│   │   └─ app.get('/') route register
│   └─ export default app
├─ import config from './config'
│   ↓
│   [src/config/index.ts]
│   │   └─ import dotenv → .env ফাইল load
│   │   └─ process.env থেকে PORT, DATABASE_URL নেয়
│   └─ export default config
├─ console.log(database_url) → verify করে
├─ app.listen(5000) → সার্ভার চালু!
└─ Node.js Event Loop → requests-এর অপেক্ষায়...

[Browser: http://localhost:5000/]
↓
[HTTP GET / request]
↓
[Express routing engine]
↓
[app.get('/') handler]
↓
[res.send("Hello world...")] → Response ব্রাউজারে ফেরত যায়

```

🎯 Next Steps — পরবর্তী সেশনে যা যোগ হতে পারে

এই ফাউন্ডেশনের উপর ভিত্তি করে সাধারণত এরপর আসে:

1. **Middleware** — `app.use(express.json())` — JSON body parsing
2. **Router** — Route modularization (`/api/users`, `/api/products`)
3. **Controller** — Business logic আলাদা ফাইলে
4. **Model/Schema** — Database schema definition
5. **Error Handling** — Global error handler middleware
6. **Authentication** — JWT/Session based auth
7. **Validation** — Zod/Joi দিয়ে input validation
8. **CORS** — Cross-Origin Resource Sharing
9. **Rate Limiting** — API abuse protection
10. **Logging** — Winston/Pino দিয়ে structured logging

💡 **মনে রাখো:** বিশ্বসেরা ইঞ্জিনিয়ার হতে হলে শুধু "কাজ করে" বুঝলে হবে না — "কেন কাজ করে", "ভেতরে কী হচ্ছে", "এটা না করলে কী হতো" — এগুলো বুঝতে হবে। তুমি সেই পথেই আছো! 🚀

এই নোটটি নতুন কোড যোগ হলে আপডেট হবে।

🔑 ৮. প্রফেশনাল গিট ও সিকিউরিটি ম্যানেজমেন্ট (Professional Git & Secrets Management)

সফটওয়্যার ইন্ডাস্ট্রিতে কোড লেখার পাশাপাশি কোড এবং সিস্টেমের সিকিউরিটি বজায় রাখা একজন বিশ্বসেরা ইঞ্জিনিয়ারের অন্যতম প্রধান দায়িত্ব।

🔒 ক) সিক্রেটস ম্যানেজমেন্ট ও কেন আমরা `.env` গিটহাবে দেব না?

- **বট স্ক্যানিং:** হ্যাকারদের স্ক্রিপ্টগুলো অনবরত গিটহাবের পাবলিক এন্ট্রি স্ক্যান করে সংবেদনশীল কি (Keys) খোঁজার জন্য। আপনি ভুলবশত পুশ করার কয়েক সেকেন্ডের মধ্যেই আপনার ক্রেডেনশিয়াল ফাস হয়ে যেতে পারে।
- **ডাটাবেজ হাইজ্যাকিং:** ডাটাবেজ লিংক পেলে ডাটা চুরি বা রেনসমওয়্যার অ্যাটাক হওয়ার শতভাগ সম্ভাবনা থাকে।
- **গিট হিস্ট্রি লকিং:** একবার কোনো কমিটে ডাটা পাস হয়ে গেলে তা গিট হিস্ট্রিতে চিরকালের জন্য সেভ হয়ে যায়, ফাইল ডিলিট করলেও সেই ওন্ড কমিট থেকে ডাটা রিকভার করা সম্ভব।

সমাধান: ১. `.gitignore` ফাইলে `.env` ফোল্ডার বা ফাইলের নাম যুক্ত করা। ২. একটি `.env.example` ফাইল কমিট করা যেখানে ভ্যারিয়েবলগুলোর নাম ডামি ভ্যালু সহ উল্লেখ থাকে।

📄 খ) প্রফেশনাল কমিট ম্যাসেজিং (Conventional Commits)

আপনার কমিট মেসেজ দেখে আপনার কোডের ম্যাচিউরিটি বোঝা যায়। আমরা সবসময় নিচের ফরম্যাটে কমিট করব:

`type(scope): description`

- **feat:** নতুন কোনো ফিচার যুক্ত করা (যেমন: `feat(db): establish neon connection`)
- **fix:** কোনো বাগ বা ত্রুটি ফিক্স করা (যেমন: `fix(auth): correct jwt token expiration`)
- **docs:** শুধুমাত্র ডকুমেন্টেশন বা README আপডেট করা (যেমন: `docs(readme): add installation guide`)
- **chore:** প্যাকেজ ইনস্টলেশন, কনফিগ পরিবর্তন বা বিন্ড টুলস আপডেট (যেমন: `chore(git): add ignore templates`)
- **refactor:** নতুন কোনো ফিচার না এনে বা বাগ ফিক্স না করে কেবল কোডের মান উন্নত করা।

🌿 গ) ব্রাঞ্চিং ও পুল রিকুয়েস্ট ফ্লো (Branching & Pull Request)

কখনো সরাসরি `main` বা `master` ব্রাঞ্চে সরাসরি কোড পুশ করা রিয়েল ওয়ার্ল্ড প্রজেক্টে কঠোরভাবে নিষিদ্ধ।

- **ফিচার ব্রাঞ্চ (Feature Branching):** প্রতিটি নতুন কাজের জন্য লোকাল ব্রাঞ্চ তৈরি করতে হয় (যেমন: `git checkout -b feat/add-user-controller`)।
 - **পিআর রিভিউ (PR Review):** ফিচার ব্রাঞ্চের কাজ গিটহাবে পুশ করার পর `main` ব্রাঞ্চের সাথে মার্জ করার জন্য একটি **Pull Request (PR)** ওপেন করা হয়। টিম মেম্বারদের কোড রিভিউ ও অ্যাপ্রভালের পর তা মার্জ হয়।
-